DOCUMENTAÇÃO FUNCIONAL DO SOFTWARE

Console de Ingestão InfoPrice

Coleta SFTP · PostgreSQL · Rastreabilidade ponta a ponta

Repositório: melhorartecnologia-arch/etlinfoproce

Componentes: server (Node.js/TypeScript) · web (React/Vite) · shared · db

Versão do software: 1.0.0 · Documento gerado em 18/10/2026

Sumário

1. Visão geral 4

Problemas que o software resolve 4

Público-alvo 4

2. Arquitetura 4

Fluxo de alto nível 5

3. Processo de coleta e ingestão 5

Um arquivo por vez, até a gravação 6

Idempotência 6

Parar uma coleta 6

Situações de uma execução 6

Gatilhos e tipos 6

4. Leitura dos arquivos 6

5. Regras de qualidade 7

6. Merge incremental e tabela final 8

7. Rastreabilidade e linhagem 8

Inventário versus participação 8

8. Agendamento automático 9

Tentativas, tolerância e pausa 9

9. Alertas e incidentes 9

10. Console web: as telas 10

Login e primeiro acesso 10

Cabeçalho e menu do usuário 10

Coleta do dia 11

Alertas e incidentes 11

Execução passo a passo 11

Linhagem do dado 12

Inventário SFTP 12

Qualidade e rejeições 12

Consulta de preços 13

Conexão e destino 13

Usuários 13

Zerar base 13

11. Segurança e controle de acesso 14

Papéis . . . 14
Autenticação e sessão . . . 14
Segredos e infraestrutura . . . 14
12. Modelo de dados . . . 15
Funções SQL . . . 15
Migrações . . . 15
13. API REST . . . 16
14. Configuração . . . 17
15. Comandos de linha de comando . . . 18
Rodando sem o servidor do fornecedor . . . 18
16. Implantação e operação . . . 19
Diagnóstico de problemas . . . 19
Atualização . . . 19
17. Glossário . . . 20

1. Visão geral

O **Console de Ingestão InfoPrice** é uma aplicação web que automatiza a coleta diária dos arquivos de pesquisa de preços entregues pela InfoPrice (produto *ISA-InfoPanel*) num servidor SFTP, grava esses dados num banco PostgreSQL e oferece **rastreabilidade de ponta a ponta**: a partir de qualquer linha de preço gravada é possível chegar ao arquivo, à linha do arquivo e à execução que a trouxe.

Além do processo de ETL (extração, transformação e carga), o software entrega um console de operação em português, com nove telas, que permite acompanhar a coleta em tempo real, investigar falhas, consultar rejeições de qualidade, reprocessar dados, gerenciar incidentes e administrar quem acessa o sistema.

Problemas que o software resolve

- **Coleta sem intervenção manual:** a pasta do dia é baixada automaticamente às 05:30, com novas tentativas e varreduras de segurança caso o fornecedor publique atrasado.
- **Retenção curta na origem:** o fornecedor mantém os arquivos por apenas 5 dias; o console guarda cópia local por 30 dias e avisa quando algo está para expirar.
- **Qualidade do dado:** oito regras automáticas barram linha malformada, GTIN inválido, preço zerado ou acima do limite, loja desconhecida, data fora da janela, duplicidade e texto com encoding quebrado.
- **Segurança da carga:** as etapas de gravação rodam numa única transação; em caso de falha, a tabela final permanece com o dado do dia anterior.
- **Auditoria:** cada execução tem log técnico, etapas cronometradas, hashes SHA-256 dos arquivos e relatório exportável em CSV e PDF.
- **Alertas:** incidentes deduplicados aparecem no painel e são enviados por e-mail.

Público-alvo

Perfil	Uso típico
Equipe de dados / engenharia	Opera a coleta, investiga falhas, reprocessa runs e arquivos.
Analistas de BI	Consultam os preços gravados, conferem rejeições e exportam CSV.
Administradores	Criam usuários, definem papéis e acompanham tentativas de acesso.
Infraestrutura	Implantam o serviço (Lightsail/RDS), configuram TLS, SMTP e segredos.

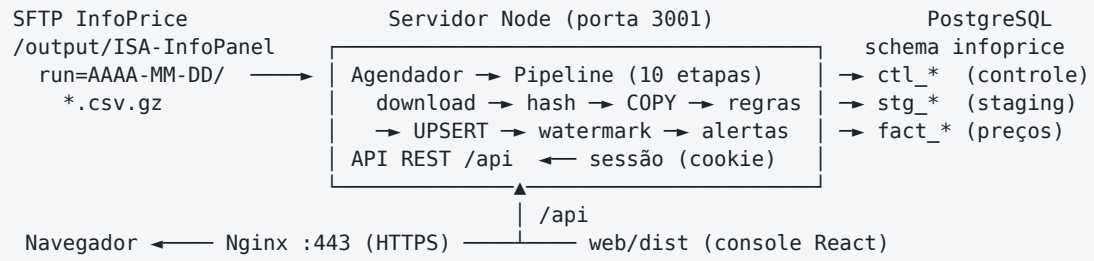
2. Arquitetura

O repositório é um *monorepo* npm com três *workspaces* e uma pasta de migrações SQL. Servidor e console compartilham os mesmos tipos de domínio, o que garante que a tela e a API falem exatamente o mesmo contrato.

Pasta	Conteúdo	Tecnologia
`server/`	Coleta SFTP, pipeline de ingestão, agendador, alertas, autenticação e API REST	Node.js 20+, TypeScript, Express, pg, pg-copy-streams, node-cron, nodemailer, pdfkit, ssh2-sftp-client
`web/`	Console de operação (as nove telas)	React 18, Vite 6, TypeScript
`shared/`	Tipos de domínio e contratos da API usados pelos dois lados	TypeScript
`db/migrations/`	Schema PostgreSQL versionado (9 migrações)	PostgreSQL 14+

Pasta	Conteúdo	Tecnologia
`deploy/`	Configuração do Nginx (proxy reverso + arquivos estáticos)	Nginx
`ecosystem.config.cjs`	Definição do processo para o PM2	PM2

Fluxo de alto nível



Em produção o Nginx atende as portas 80/443, serve o console como arquivos estáticos e repassa `/api` para o Node na porta 3001, que nunca é exposto diretamente à internet. O processo roda em **uma única instância**: o "trilho" que impede duas coletas simultâneas vive na memória do processo.

3. Processo de coleta e ingestão

A coleta de um dia (um *run*) corresponde a uma pasta `run=AAAA-MM-DD` no diretório base do SFTP. Cada execução percorre dez etapas, exatamente as mesmas exibidas no painel, cada uma com horário de início, duração, detalhe e situação (pendente, em curso, ok, erro ou ignorada).

#	Etapa	O que faz
1	Conexão SFTP	Abre a conexão com a origem. Autenticação preferencial por chave privada `.pem`; senha só como alternativa.
2	Listagem do diretório	Localiza a pasta do run, lista os arquivos, registra cada um no inventário (`ctl_arquivo`) e compara a quantidade com a média dos últimos 7 dias.
3	Download para área temporária	Baixa os arquivos em paralelo (padrão: 4 simultâneos) para `var/spool/infoprice/<run>`.
4	Verificação de integridade	Calcula o SHA-256 do arquivo comprimido e do conteúdo descompactado (gunzip) e grava ambos.
5	Validação de schema	Antes de carregar qualquer linha, confere se o cabeçalho de cada arquivo traz as colunas obrigatórias: gtin, descricao, id_loja, preco, data_coleta. Arquivo fora do layout é recusado em segundos.
6	Carga em staging	Arquivo a arquivo: apaga o staging do arquivo e o recarrega via `COPY` em `stg_isa_infopanel_preco`, tudo como texto.
7	Regras de qualidade	Aplica as oito regras às linhas do arquivo; cada linha reprovada vai para `ctl_rejeicao` com o payload original.
8	Merge incremental	`UPSERT` do arquivo em `fact_preco_coletado` pela chave (gtin, id_loja, data_coleta, fonte) e `COMMIT` antes do próximo arquivo.
9	Auditoria e watermark	Grava as contagens por arquivo e avança `ctl_watermark` (data do último run efetivado).
10	Notificação	Avalia as regras de alerta, abre incidentes e envia o resumo por e-mail e painel.

Um arquivo por vez, até a gravação

As etapas 1 a 5 acontecem fora de transação (rede, disco e leitura do cabeçalho). As etapas 6 a 8 rodam **arquivo a arquivo, cada um numa transação própria**: o arquivo passa por staging, regras e merge e é efetivado antes de o próximo começar. O que já foi gravado fica gravado — uma falha no arquivo 30 reverte apenas o arquivo 30; os demais seguem, a execução fecha como parcial e a varredura tenta o arquivo de novo.

O painel acompanha o avanço a cada arquivo, e o log registra uma linha por arquivo gravado (lidas, rejeitadas, inseridas, atualizadas e duração). O log técnico é gravado fora das transações e sobrevive para explicar qualquer falha.

Cada transação de arquivo tem teto próprio de tempo (`PG_TIMEOUT_INGESTAO_MS`, 1 hora por padrão), separado do limite de 30 segundos aplicado às consultas das telas.

Idempotência

Reexecutar o mesmo run é seguro: o staging de cada arquivo é apagado e recarregado, e o UPSERT só atualiza a linha cujo preço mudou de fato (`WHERE f.preco IS DISTINCT FROM EXCLUDED.preco`). Reprocessar um dia sem alteração na origem grava zero linhas e não infla as contagens.

Parar uma coleta

Uma coleta em andamento pode ser parada pelo botão **Parar coleta**, por Ctrl+C no terminal ou pelo desligamento do servidor. Ela para num ponto seguro — entre arquivos, a cada lote lido, e a consulta SQL em curso é interrompida no próprio PostgreSQL. O pedido fica gravado no banco, então alcança também uma coleta rodando em outro processo.

A coleta grava um batimento a cada 30 segundos. Uma execução em andamento sem batimento há mais de 2 minutos é **órfã** — o processo morreu sem encerrá-la — e é encerrada como cancelada na subida do servidor, antes de uma coleta nova, antes de zerar a base ou pelo botão Parar coleta. Sem isso, como só uma execução pode estar em andamento, ela bloquearia todas as seguintes.

Situações de uma execução

Status	Significado
em_execucao	Coleta em andamento. Só pode existir uma por vez (índice único no banco).
concluida	Todos os arquivos vistos na origem foram ingeridos.
parcial	Parte dos arquivos ficou de fora; os pendentes são retomados pela varredura.
falha	Nenhum arquivo pôde ser gravado (ex.: conexão, pasta ausente ou layout recusado em todos).
cancelada	Parada a pedido (botão Parar coleta, Ctrl+C ou desligamento do servidor), ou órfã encerrada. Arquivos já gravados ficam; o arquivo em curso é revertido.

Gatilhos e tipos

Cada execução registra **o que a disparou** — *agendador*, *manual* (botão na tela), *varredura*, *retentativa* ou *reprocessamento* — e o **tipo de carga**: *incremental*, *reprocessamento* ou *carga histórica*. Quando disparada pela tela, guarda também o usuário responsável.

4. Leitura dos arquivos

Os arquivos chegam como `.csv.gz` (ou `.csv`) com 44 colunas (layout ISA-InfoPanel). A leitura é feita em *streaming*, em lotes de 5.000 linhas: um arquivo de 160 MB comprimido nunca é carregado inteiro em memória, e a leitura anda no ritmo do COPY para o banco.

- **Delimitadores aceitos:** barra vertical (`|`, o do arquivo da InfoPrice), ponto e vírgula, vírgula ou tabulação, detectado pelo cabeçalho; BOM é ignorado. Nulo gravado como `\N` (padrão do Spark) é lido como vazio.
- **Layout InfoPrice:** `data_preco` → data_coleta; `loja` → id_loja (a coluna `id_loja` do arquivo só cobre a ausência dela); `cidade` → municipio; `preco_regular` → preco; `preco_pago` vira o preço promocional quando `flag_promocao` = true, que também define o tipo de preço (Promoção ou Regular).
- **Normalização do cabeçalho:** minúsculas, sem acentos, espaços viram "_".
- **Sinônimos:** variações da origem são absorvidas — por exemplo *ean*, *codigo_barras* → gtin; *produto* → descricao; *codigo_loja* → id_loja; *valor*, *preco_venda* → preco; *dt_coleta*, *data_pesquisa* → data_coleta; *bandeira* → rede; *estado* → uf; *cidade* → municipio.
- **Payload preservado:** a linha original inteira é guardada no campo `payload`, de modo que nenhuma das 44 colunas se perde na rastreabilidade e a tela de rejeições pode mostrar o dado como veio.
- **Conversões tolerantes:** números aceitam vírgula decimal ("1.234,56"), datas aceitam ISO (AAAA-MM-DD) e dd/mm/aaaa. O que não converte vira rejeição rastreável em vez de derrubar a carga.
- **Aspas tolerantes:** aceita os escapes `""` e `\"` (padrão do Spark, que gera os arquivos `part-*.csv.gz`) e campos entre aspas com quebra de linha. Uma aspa solta é lida como texto quando a linha mantém o número certo de colunas; quando não mantém, a linha entra no staging com o texto original e é rejeitada como "Linha malformada no arquivo". Uma linha defeituosa nunca interrompe a execução.
- **Layout configurável:** nomes de coluna diferentes são mapeados em `MAPA_COLUNAS` (ex.: `gtin=cd_ean,preco=vl_preco`); arquivos sem linha de cabeçalho — o padrão do Spark — são lidos com `CSV_COLUNAS`, que lista os nomes na ordem das colunas. O comando `inspecionar-arquivo` mostra como um arquivo é interpretado.

5. Regras de qualidade

As regras rodam em ordem sobre o staging. A primeira regra que reprova uma linha é a "dona" da rejeição, portanto uma linha nunca aparece com dois motivos. Cada rejeição registra arquivo, número da linha, motivo, payload original e o tratamento previsto.

#	Regra	Quando reprova	Tratamento registrado
1	Linha malformada no arquivo	A linha não pôde ser separada em colunas (aspas desbalanceadas com contagem de colunas divergente do cabeçalho).	Aguardando correção na origem
2	GTIN inválido no dígito verificador	GTIN não numérico, com comprimento diferente de 8, 12, 13 ou 14, ou dígito verificador divergente (módulo 10 GS1).	Aguardando correção na origem
3	Preço menor ou igual a zero	Preço ausente, não numérico ou ≤ 0.	Descartada por regra
4	Preço acima do limite	Preço ou preço promocional a partir de 10 bilhões, além do que numeric(12,2) comporta; estouraria o merge do arquivo.	Aguardando correção na origem

#	Regra	Quando reprova	Tratamento registrado
5	Loja ausente em dim_loja	Loja vazia, ou não cadastrada na dimensão de lojas. Com CADASTRAR_LOJAS (padrão), as lojas do arquivo são cadastradas antes, com rede, UF e cidade, e só a linha sem loja é rejeitada.	Cadastro de loja solicitado
6	Data de coleta fora da janela do run	Data inválida, posterior ao run ou mais antiga que JANELA_COLETA_DIAS (365 dias por padrão: o arquivo traz o último preço visto em cada loja).	Reenfileirada para run correto
7	Duplicidade na chave de conflito	Mesma combinação gtin + loja + data + fonte repetida dentro do arquivo; mantém a última ocorrência. Entre arquivos do run, vira atualização no merge.	Última ocorrência mantida
8	Caractere inválido na descrição	Sequência "\xNN" literal, caractere de controle ou caractere de substituição U+FFFD (encoding quebrado).	Normalizada e reprocessada

Se as rejeições ultrapassarem **0,5%** das linhas do run (`LIMITE_REJEICOES_PCT`), um incidente de severidade "Atenção" é aberto.

6. Merge incremental e tabela final

Somente as linhas aprovadas pelas regras de qualidade são promovidas do staging para `fact_preco_coletado`. A chave de conflito é **(gtin, id_loja, data_coleta, fonte)**. Linha nova é inserida; linha existente com preço diferente é atualizada; linha idêntica é ignorada.

A tabela final é **particionada por mês** em `data_coleta`. Antes do merge, a função `garantir_particao` cria automaticamente as partições mensais necessárias (ex.: `fact_preco_coletado_2026_08`), considerando apenas datas de linhas válidas — uma data absurda de um arquivo defeituoso não cria partição.

O comando de persistência exibido na tela de detalhe da execução é a mesma constante usada pelo pipeline, de modo que a tela nunca mostra um SQL diferente do que roda de fato:

```
INSERT INTO infoprice.fact_preco_coletado AS f
SELECT * FROM infoprice.stg_isa_infopanel_preco
WHERE run_date = DATE '{run}'
ON CONFLICT (gtin, id_loja, data_coleta, fonte)
DO UPDATE SET preco = EXCLUDED.preco,
              id_execucao = EXCLUDED.id_execucao,
              atualizado_em = now()
WHERE f.preco IS DISTINCT FROM EXCLUDED.preco;
```

Toda linha da tabela final carrega `id_execucao`, `id_arquivo` e `numero_linha`, além de `criado_em` e `atualizado_em` — a base da rastreabilidade inversa.

7. Rastreabilidade e linhagem

A rastreabilidade é o eixo do produto. Ela funciona nos dois sentidos:

- **Do arquivo para o dado:** para cada arquivo de uma execução, quantas linhas havia no arquivo, quantas entraram no staging, quantas foram rejeitadas, inseridas e atualizadas, e em qual partição foram gravadas.
- **Do dado para o arquivo (caminho inverso):** a partir de uma linha de `fact_preco_coletado`, chega-se à execução, ao arquivo, à linha do arquivo, ao hash e ao caminho remoto de origem.

Inventário versus participação

`ctl_arquivo` responde "o que existe na origem": identidade *pasta* + *nome*, hash, tamanho, caminho local e prazo de retenção. Já as contagens "quantas linhas este arquivo rendeu" são propriedade de cada execução e ficam em `ctl_execucao_arquivo`. Por isso reprocessar um arquivo gera contagens novas **sem apagar o histórico** das execuções anteriores.

8. Agendamento automático

O agendador interno registra três rotinas, todas no fuso America/Sao_Paulo:

Rotina	Padrão	Variável	O que faz
Coleta diária	05:30 todos os dias	`CRON_COLETA`	Dispara a coleta do run do dia, com até 3 tentativas.
Varredura de segurança	12:00 e 18:00	`CRON_VARREDURA`	Tenta de novo a coleta do dia se ainda não houver sucesso, retoma pastas atrasadas ou pendentes dentro da retenção e verifica arquivos prestes a expirar.
Resumo diário	06:00	`CRON_RESUMO`	Envia por e-mail o resumo da execução do dia e a contagem de incidentes abertos.

Tentativas, tolerância e pausa

- **Tentativas:** 3, com espera de 10, 20 e 40 minutos entre elas (`TENTATIVAS`, `ESPERA_TENTATIVAS_MIN`). As tentativas seguintes são registradas com gatilho "retentativa".
- **Janela de tolerância:** até as 09:00 (`TOLERANCIA_ATE`) a ausência da pasta do dia não abre incidente; depois disso, abre um incidente crítico.
- **Pausa:** o operador pode pausar o agendamento pela tela. Isso impede a coleta automática — inclusive entre uma tentativa e outra — e as varreduras, mas não bloqueia a coleta manual. Fica registrado quem pausou e quando.
- **Uma coleta por vez:** agendador, varredura e tela compartilham o mesmo trilho. Uma segunda solicitação recebe HTTP 409 ("já existe uma coleta em andamento"), mensagem exibida na tela como aviso.
- **Próxima execução:** calculada a partir do cron e do fuso e exibida no cabeçalho do console.

9. Alertas e incidentes

As regras de alerta ficam na tabela `ctl_regra_alerta` e podem ser editadas no banco (condição, severidade, canal, destinatário, ativa/inativa). O painel é sempre canal obrigatório; o e-mail sai quando há SMTP configurado. Sem SMTP, o alerta continua valendo pelo painel.

Regra	Severidade	Canal padrão	Quando dispara
Falha de conexão ou execução interrompida	Crítico	e-mail + painel	Execução termina em falha. Execução parcial abre o mesmo tipo com severidade Atenção.
Pasta do dia ausente na origem às 09:00	Crítico	e-mail + painel	Varredura após a janela de tolerância não encontra a pasta. Fecha sozinho quando a pasta aparece.
Rejeições acima de 0,5% das linhas do run	Atenção	e-mail	Percentual rejeitado supera o limite configurado.
Volume fora de 25% da média de 7 dias	Atenção	e-mail	Linhas gravadas desviam mais de 25% da média das execuções concluídas nos 7 dias anteriores.

Regra	Severidade	Canal padrão	Quando dispara
Arquivo a menos de 24h da expiração	Informativo	painel	Arquivos que saem do servidor do fornecedor até amanhã; indica se já há cópia local conferida.
Resumo diário às 06:00	Informativo	e-mail	Resumo do dia para equipe de dados e BI.

- **Deduplicação:** incidentes são únicos por condição + run. Uma coleta que falha três vezes abre um incidente, não três.
- **Código legível:** cada incidente recebe um código sequencial (#201, #202, …) usado em conversas, e-mails e na tela.
- **Resolução auditável:** ao resolver, ficam registrados quem resolveu (usuário real), quando e a duração do incidente.
- **E-mail de resultado:** ao fim de cada execução concluída ou parcial é enviado um resumo (arquivos, linhas, inseridas, atualizadas, rejeições, duração). Em caso de falha, apenas o e-mail do incidente é enviado, para não duplicar.

10. Console web: as telas

O console é organizado numa barra lateral com cinco grupos: **operação**, **rastreabilidade**, **dados**, **configuração** e — apenas para administradores — **administração**. A interface esconde o que o papel do usuário não alcança, mas quem garante a permissão é o servidor.

Login e primeiro acesso

Menu: — (tela inicial) · Acesso: público

Porta de entrada do console. Nenhuma rota de dados responde sem sessão, nem para leitura.

O que a tela mostra

- Formulário de login e senha.
- Quando a senha é provisória (definida por um administrador), a tela "Defina a sua senha" exige a troca antes de qualquer outra ação.
- Mensagens de bloqueio após tentativas excessivas.

Cabeçalho e menu do usuário

Menu: presente em todas as telas · Acesso: todos (ações: operador)

Barra superior com o estado do agendamento e os atalhos de operação.

O que a tela mostra

- Diretório base do SFTP monitorado.
- "Próxima coleta dd/mm, hh:mm BRT" ou "Agendamento pausado".
- Menu do usuário com nome e papel.

Ações disponíveis

- **Densidade compacta/confortável:** alterna a altura das linhas das tabelas.
- **Pausar / Retomar agendamento** (operador).
- **Coleta manual** (operador): dispara a coleta do run do dia em segundo plano; o painel acompanha o progresso.

- **Parar coleta** (operador): aparece no lugar de Coleta manual enquanto uma coleta roda. Para num ponto seguro: os arquivos já gravados ficam, o arquivo em curso é revertido, e a execução termina como cancelada, sem incidente nem nova tentativa.
- **Trocar senha** e **Sair**, no menu do usuário.

Coleta do dia

Menu: operação › Coleta do dia · Acesso: todos

Painel principal de acompanhamento. Enquanto houver coleta em andamento, a tela se atualiza sozinha a cada 4 segundos para as etapas avançarem sem recarregar.

O que a tela mostra

- Quatro indicadores (KPIs): situação da execução de hoje e janela de horário; arquivos ingeridos e volume; linhas persistidas; rejeições.
- Aviso contextual (ex.: pasta ainda não publicada, falha, rejeições altas).
- As 10 etapas do processo, com horário, detalhe, situação e duração de cada uma.
- Histórico das últimas execuções: run, janela, tipo, arquivos ingeridos/vistos, linhas gravadas, rejeições, duração e status.

Ações disponíveis

- Clicar numa linha do histórico abre a tela de execução passo a passo.

Alertas e incidentes

Menu: operação › Alertas e incidentes · Acesso: todos (resolver: operador)

Central de incidentes. Cada incidente guarda a execução de origem, o que foi feito e quem encerrou.

O que a tela mostra

- Lista de incidentes com código, severidade, título, detalhe, horário de abertura, run, canal e situação (aberto/resolvido, com responsável e duração).
- Tabela das regras de notificação: condição, severidade, canal e destinatário.

Ações disponíveis

- **Resolver** um incidente aberto (operador), registrando o usuário responsável.
- Abrir a execução relacionada.

Execução passo a passo

Menu: rastreabilidade › Execução passo a passo · Acesso: todos (ações: operador)

Raio-X de uma execução. Sem execução escolhida, abre a mais recente.

O que a tela mostra

- Cartões-resumo da execução e frase descritiva do resultado.
- Log técnico completo (`ctl_execucao_log`) com hora, nível (DEBUG, INFO, WARN, ERRO) e mensagem.
- Controle de idempotência: assinatura, watermark anterior e novo, tentativa etc.
- Comando de persistência (SQL do UPSERT), que pode ser mostrado ou ocultado.
- Arquivos da execução: nome, tamanho, SHA-256, linhas lidas, gravadas e rejeitadas, destino e status.
- Resumo: status, gatilho, linhas em staging, gravadas, inseridas, atualizadas, rejeitadas, bytes baixados e erro, se houver.

Ações disponíveis

- **Auditoria CSV** e **Auditoria PDF**: relatório de auditoria da execução.
- **Reprocessar run** (operador): refaz a coleta do mesmo dia.
- **Baixar bruto** (operador): baixa a cópia local do arquivo, idêntica à da origem; desabilitado quando não há cópia local.
- **Reprocessar** um arquivo específico (operador).

Linhagem do dado

Menu: rastreabilidade › Linhagem do dado · Acesso: todos

Mostra o caminho do arquivo recebido até a linha gravada, com a contagem de cada passagem.

O que a tela mostra

- Fluxo visual por etapa (arquivos → staging → rejeições → fato), com objeto, quantidade e nota; cores indicam neutro, atenção ou ok.
- Contagem por arquivo: linhas no arquivo, no staging, rejeitadas, inseridas, atualizadas, partição de destino e execução.
- Caminho inverso: pega uma linha real de `fact_preco_coletado` e mostra execução, arquivo, linha, hash e origem.

Ações disponíveis

- Clicar no número da execução abre o detalhe dela.

Inventário SFTP

Menu: rastreabilidade › Inventário SFTP · Acesso: todos (ações: operador)

Controle das pastas run= vistas na origem e de sua janela de retenção (5 dias no fornecedor).

O que a tela mostra

- Cartões: pastas na origem, arquivos pendentes, quantos expiram nas próximas 24h.
- Tabela de pastas monitoradas: pasta, conteúdo (arquivos e bytes), visto em, ingerido em, retenção restante na origem e situação.

Ações disponíveis

- **Baixar a pasta** inteira em `.tar` a partir das cópias locais (operador).
- **Reprocessar a pasta** (operador).

Qualidade e rejeições

Menu: rastreabilidade › Qualidade e rejeições · Acesso: todos

Mostra o que as regras de qualidade barraram no run.

O que a tela mostra

- Execução de origem e percentual rejeitado.
- Quantidade de rejeições por motivo (as oito regras).
- Linhas rejeitadas: arquivo e linha, motivo, payload original e tratamento.

Ações disponíveis

- **Exportar rejeições (CSV)**.

Consulta de preços

Menu: dados › Consulta de preços · Acesso: todos

Leitura direta de `fact_preco_coletado`. Toda linha aponta para a execução e o arquivo que a originaram.

O que a tela mostra

- Filtros: data da coleta, UF, busca por GTIN ou descrição e tipo de preço (Todos, Regular, Promoção). Os filtros só valem ao clicar em "Buscar".
- Resultado: GTIN, descrição, loja, UF, preço, preço promocional, tipo, data de coleta e origem.

Ações disponíveis

- **Buscar** e **Exportar CSV** com os filtros aplicados.
- Clicar na origem de uma linha abre a execução que a gravou.

Conexão e destino

Menu: configuração › Conexão e destino · Acesso: todos (somente leitura)

Exibe a configuração vigente. Credenciais ficam no cofre de segredos; a aplicação recebe apenas a referência.

O que a tela mostra

- **Origem SFTP:** host, porta, usuário, autenticação, referência da senha no cofre (`vault/infoprice/sftp`), diretório base, padrão de pasta, retenção.
- **Agendamento e tolerância:** cron da coleta, tentativas, janela de tolerância, varredura, paralelismo de download, alerta de volume.
- **Destino PostgreSQL:** instância, banco e schema, tabelas de staging e fato, chave de conflito e estratégia de carga.
- **Alertas:** regras ativas e seus canais.

Usuários

Menu: administração › Usuários · Acesso: administrador

Gestão de quem entra no console, com qual papel, e auditoria de acesso.

O que a tela mostra

- Lista de usuários: login, nome, papel, situação (ativo, desativado, senha provisória), último acesso e sessões abertas.
- Tentativas de acesso: quando, login, origem (IP), resultado e motivo.

Ações disponíveis

- **Criar usuário** (login, nome, e-mail opcional, papel); a senha provisória é exibida uma vez.
- **Alterar papel**, nome ou situação; **desativar/ativar**.
- **Redefinir senha**: gera nova senha provisória e encerra as sessões do usuário.
- O último administrador ativo não pode ser rebaixado nem desativado.

Zerar base

Menu: administração › Zerar base · Acesso: administrador

Apaga todo o histórico de dados e de metadados e deixa a aplicação como recém-instalada, mantendo os usuários. Roda numa transação só: ou apaga tudo, ou nada.

O que a tela mostra

- O que será apagado, com a contagem atual: execuções, etapas e log; inventário de arquivos; preços gravados e todas as partições mensais; staging e rejeições; incidentes (o código volta a #201); lojas; tentativas de login.
- O que fica: usuários e sessões (quem zera continua conectado), regras de alerta, agendamento, schema e o registro dos resets anteriores (quem, quando, de onde).
- Aviso quando há coleta em andamento: nesse caso o reset é recusado, e nenhuma coleta começa enquanto ele roda.

Ações disponíveis

- **Apagar também as cópias locais** dos arquivos baixados (opcional). O fornecedor guarda só 5 dias: o que for mais antigo não poderá ser baixado de novo.
- **Zerar a base**: exige digitar `ZERAR TUDO` e a senha de quem pede; o servidor confere as duas.
- Apagar também os usuários só pelo terminal: `zerar-base --incluir-usuarios`.

11. Segurança e controle de acesso

Papéis

Papel	Pode
leitor	Ver todas as telas e exportar relatórios.
operador	O acima, mais coleta manual, reprocessamento (run, arquivo, pasta), pausar/retomar agendamento, resolver incidentes e baixar arquivos.
administrador	O acima, mais a gestão de usuários, a auditoria de acesso e zerar a base.

Autenticação e sessão

- **Sem administrador padrão:** uma instalação nova não tem usuário; o primeiro administrador é criado pela linha de comando, com senha sorteada exibida uma única vez.
- **Sessão no servidor:** cookie `HttpOnly` + `SameSite=Strict` (e `Secure` em produção). O banco guarda apenas o SHA-256 do token. Duração padrão: 12 horas.
- **Revogação imediata:** desativar um usuário ou redefinir sua senha encerra as sessões na hora, sem esperar expirar.
- **Senhas:** `scrypt` (N=16384) com sal por usuário e comparação em tempo constante; mínimo de 10 caracteres misturando letras e números.
- **Senha provisória:** definida por administrador, obriga a troca no primeiro acesso.
- **Força bruta:** 5 falhas em 15 minutos travam o usuário; o limite por IP é 4 vezes maior, para não punir um escritório atrás do mesmo NAT. Toda tentativa fica em `ctl_tentativa_login`.
- **Autoria:** execuções manuais, pausas de agendamento e resoluções de incidente ficam vinculadas ao usuário real.

Segredos e infraestrutura

- Valores no repositório são *placeholders*; os reais vivem só no `server/.env` (ignorado pelo git), assim como as chaves `*.pem`.

- A senha do SFTP nunca é exibida; a tela mostra apenas a referência do cofre.
- Conexão ao RDS com TLS `verify-full` (certificado e hostname). O health check `/api/saude` informa se a conexão está cifrada de fato.
- Credencial do banco por senha, AWS Secrets Manager ou token IAM de 15 minutos.

12. Modelo de dados

Todas as tabelas ficam no schema `infoprice`, com a convenção de prefixos `ctl_` (controle e rastreabilidade), `stg_` (staging), `dim_` (dimensão) e `fact_` (dado final).

Tabela	Finalidade
`ctl_execucao`	Uma linha por execução: gatilho, tipo, início, fim, status, contagens, watermark, tentativa, erro e usuário.
`ctl_execucao_etapa`	As 10 etapas de cada execução, com duração individual.
`ctl_execucao_log`	Log técnico em ordem cronológica, com nível e mensagem.
`ctl_arquivo`	Inventário: cada arquivo visto na origem (pasta + nome), hashes, tamanho, caminho local, retenção.
`ctl_execucao_arquivo`	Participação de um arquivo numa execução: contagens e horários daquela passagem.
`ctl_rejeicao`	Linha rejeitada, motivo, payload original e tratamento.
`ctl_incidente`	Incidentes: código, severidade, título, detalhe, run, canal, resolução e responsável.
`ctl_regra_alerta`	As seis regras de notificação, editáveis.
`ctl_agendamento`	Linha única com estado de pausa, cron, fuso e quem pausou.
`ctl_watermark`	Último run efetivado por fonte.
`ctl_usuario`	Usuários, papel, hash de senha, situação e obrigação de troca.
`ctl_sessao`	Sessões do lado do servidor (hash do token, expiração, IP, agente).
`ctl_tentativa_login`	Auditoria de tentativas de login, com sucesso ou não.
`ctl_reset`	Cada vez que a base foi zerada: quem, quando, opções e o que havia antes. Não é apagada pelo reset.
`dim_loja`	Cadastro de lojas (rede, nome, UF, cidade); usado pela regra 3.
`stg_isa_infopanel_preco`	Espelho do arquivo recebido, tudo como texto, apagado e recarregado por run.
`fact_preco_coletado`	Preço final, particionado por mês, com execução, arquivo e linha de origem.

Funções SQL

Função	O que faz
`gtin_valido(text)`	Valida GTIN-8/12/13/14 pelo dígito verificador módulo 10 GS1.
`para_numero(text)`	Converte texto em número aceitando vírgula decimal; NULL se inválido.
`para_data(text)`	Converte texto ISO ou dd/mm/aaaa em data; NULL se inválido.
`garantir_particao(date)`	Cria a partição mensal da tabela fato, se ainda não existir.

Migrações

Arquivo	Conteúdo
`001_schema.sql`	Estrutura base: controle, staging, fato particionado, dimensão e funções.
`002_regras_alerta.sql`	As seis regras de notificação padrão.
`003_sequencia_incidente.sql`	Sequência própria do código do incidente (a partir de #201).
`004_execucao_arquivo.sql`	Separa inventário de arquivo da participação em cada execução.
`005_usuarios.sql`	Usuários, sessões, tentativas de login e autoria nas ações.
`006_erro_leitura_staging.sql`	Coluna erro_leitura no staging: linha malformada vira rejeição, não falha.
`007_indices_por_arquivo.sql`	Índices para o processamento arquivo a arquivo (staging e rejeições).
`008_registro_reset.sql`	Tabela ctl_reset: registro de quem zerou a base.
`009_cancelamento_execucao.sql`	Batimento e pedido de parada em ctl_execucao: parar coletas e detectar órfãs.

As migrações rodam automaticamente na subida do servidor, protegidas por *advisory lock* (várias instâncias subindo juntas não aplicam o mesmo arquivo em paralelo) e sem `statement_timeout`. O schema não exige superusuário.

13. API REST

Todas as rotas ficam sob `/api` e, exceto login e health check, exigem sessão. Rotas de ação exigem papel de operador; gestão de usuários exige administrador.

Método	Rota	Finalidade	Papel
GET	/api/saude	Health check: banco, versão, TLS efetivo, pool, latência	público
POST	/api/sessao	Login	público
GET	/api/sessao	Usuário logado	sessão
DELETE	/api/sessao	Logout	sessão
POST	/api/sessao/senha	Troca da própria senha	sessão
GET	/api/painel	KPIs, etapas e histórico da tela do dia	leitor
GET	/api/execucoes/:id	Detalhe, log, idempotência, arquivos	leitor
GET	/api/execucoes/:id/linhagem	Fluxo e contagem por arquivo	leitor
GET	/api/linhagem/preco/:id	Caminho inverso de uma linha do fato	leitor
GET	/api/inventario	Pastas monitoradas e retenção	leitor
GET	/api/qualidade	Motivos e linhas rejeitadas	leitor
GET	/api/incidentes	Incidentes e regras de notificação	leitor
GET	/api/precos	Consulta do fato com origem de cada linha	leitor
GET	/api/config	Conexão, agendamento, destino e tabelas	leitor
GET	/api/execucoes/:id/auditoria.csv | .pdf	Relatório de auditoria	leitor
GET	/api/qualidade/rejeicoes.csv	Exportação de rejeições	leitor
GET	/api/precos.csv	Exportação de preços	leitor
POST	/api/execucoes	Coleta manual (responde 202)	operador
POST	/api/execucoes/:id/cancelar	Para a coleta em andamento	operador

Método	Rota	Finalidade	Papel
POST	/api/execucoes/:id/reprocessar	Reprocessa o run	operador
POST	/api/arquivos/:id/reprocessar	Reprocessa um arquivo	operador
POST	/api/inventario/:pasta/reprocessar	Reprocessa a pasta	operador
POST	/api/agendamento/pausar | retomar	Pausa e retoma o agendamento	operador
POST	/api/incidentes/:codigo/resolver	Marca incidente como resolvido	operador
GET	/api/arquivos/:id/download	Arquivo bruto (cópia local)	operador
GET	/api/inventario/:pasta/download	Pasta inteira em .tar	operador
GET	/api/usuarios	Lista de usuários	administrador
POST	/api/usuarios	Cria usuário	administrador
PATCH	/api/usuarios/:id	Edita papel, nome ou situação	administrador
POST	/api/usuarios/:id/senha	Redefine senha	administrador
GET	/api/usuarios/tentativas	Auditoria de acesso	administrador
GET	/api/admin/base	O que um reset apagaria	administrador
POST	/api/admin/base/zerar	Zera a base (frase + senha)	administrador

14. Configuração

Toda a configuração vem de variáveis de ambiente no arquivo `server/.env` (modelo em `server/.env.example`).

Grupo	Variáveis principais	Observações
Servidor	`PORT, NODE_ENV`	API na porta 3001 por padrão.
Autenticação	`SESSAO_DURACAO_HORAS, LOGIN_MAX_TENTATIVAS, LOGIN_JANELA_MIN, COOKIE_SEGURO`	12 h; 5 tentativas em 15 min; cookie Secure obrigatório sob HTTPS.
PostgreSQL	`PGHOST, PGPORT, PGDATABASE, PGUSER, PGSCHEMA, PGPOOL_MAX, INSTANCIA_ROTULO`	Em RDS, usar o endpoint do writer.
TLS	`PGSSLMODE, CA_RDS`	verify-full automático para hosts .rds.amazonaws.com.
Credencial	`PG_CREDENCIAL, PGPASSWORD, PG_SEGREDO_ARN, AWS_REGION`	env, secrets-manager ou iam.
Tempos do banco	`PG_TIMEOUT_CONEXAO_MS, PG_TIMEOUT_CONSULTA_MS, PG_TIMEOUT_INGESTAO_MS, PG_TIMEOUT_TRANSACAO_OCIOSA_MS`	30 s para telas; 1 h para a ingestão; derruba transação ociosa.
Origem SFTP	`SFTP_DRIVER, SFTP_HOST, SFTP_PORT, SFTP_USUARIO, SFTP_CHAVE_PRIVADA, SFTP_SENHA, SFTP_DIRETORIO_BASE, SFTP_DIRETORIO_LOCAL`	driver local (diretório do disco) ou sftp (servidor real).
Coleta	`AREA_TEMPORARIA, DOWNLOAD_PARALELISMO, RETENCAO_ORIGEM_DIAS, RETENCAO_LOCAL_DIAS, COLUNAS_ESPERADAS, FONTE, MAPA_COLUNAS, CSV_COLUNAS, JANELA_COLETA_DIAS, CADASTRAR_LOJAS`	Layout do arquivo em MAPA_COLUNAS/CSV_COLUNAS; janela de datas; cadastro automático de lojas. Spool local; 4 downloads simultâneos; 5 dias na origem, 30 locais.

Grupo	Variáveis principais	Observações
Agendamento	`AGENDAMENTO_ATIVO, CRON_COLETA, CRON_VARREDURA, CRON_RESUMO, TIMEZONE, TENTATIVAS, ESPERA_TENTATIVAS_MIN, TOLERANCIA_ATE, DESVIO_VOLUME_PCT, LIMITE_REJEICOES_PCT`	Ver seção 8.
Alertas	`SMTP_HOST, SMTP_PORT, SMTP_USUARIO, SMTP_SENHA, SMTP_REMETENTE, ALERTA_DESTINATARIOS, RESUMO_DESTINATARIOS`	Sem SMTP_HOST, os alertas ficam só no painel.

15. Comandos de linha de comando

Executados com `npm run <comando> --workspace @infoprice/server`, salvo indicação.

Comando	Função
`usuario criar|listar|senha|papel|desativar|ativar`	Gestão de usuários pelo terminal; usado para criar o primeiro administrador.
`zerar-base [-- --apagar-arquivos --incluir-usuarios --sim]`	Apaga todo o histórico de dados e metadados. Com --incluir-usuarios, também usuários e sessões (estado de instalação). Pede a frase de confirmação; pare o servidor antes.
`migrate`	Aplica as migrações do schema.
`seed`	Popula a dimensão de lojas (dim_loja).
`ingest -- --run AAAA-MM-DD`	Executa a coleta de um run pela linha de comando. Ctrl+C para a coleta num ponto seguro; um segundo Ctrl+C, depois de 2 s, encerra na hora.
`inspecionar-arquivo --`	Mostra delimitador, cabeçalho, o mapeamento de cada coluna e o que falta; sem argumento, usa o arquivo mais recente da área temporária.
`gerar-fixtures -- --dias --arquivos --linhas --semente`	Gera arquivos .csv.gz de exemplo no formato InfoPrice, com ~0,32% de defeitos semeados (um por regra) e aspas escapadas no estilo Spark. --semente muda os preços para demonstrar atualização no merge.
`testar-conexao`	Confere a conexão ao banco e se ela está cifrada de fato (pg_stat_ssl); falha se o banco remoto estiver em texto claro.
`baixar-ca-rds`	Baixa o bundle de certificados de CA do Amazon RDS.
`baixar-fontes`	Baixa as fontes do console para servir localmente, sem depender de CDN.
`npm run dev (raiz)`	Sobe API em :3001 e console em :5173 para desenvolvimento.
`npm run typecheck / build (raiz)`	Verificação de tipos e compilação de servidor e console.

Rodando sem o servidor do fornecedor

Com `SFTP_DRIVER=local` (padrão), a origem é um diretório do disco com a mesma interface do SFTP real; o pipeline inteiro roda — download, hash, COPY, UPSERT.

```
npm install
cp server/.env.example server/.env        # ajuste as credenciais
npm run migrate && npm run seed
npm run gerar-fixtures --workspace @infoprice/server -- --dias 6 --arquivos 4 --linhas 5000
npm run ingest --workspace @infoprice/server -- --run 2026-08-25
npm run dev                               # API :3001, console :5173
```

16. Implantação e operação

O guia `DEPLOY.md` descreve a publicação num servidor Ubuntu do Amazon Lightsail com PM2, Nginx e certificado Let's Encrypt, usando banco no Amazon RDS. Resumo dos passos:

Passo	Ação
1	Criar instância Ubuntu 24.04 (mínimo 2 GB RAM), IP estático e firewall (22 restrito, 80/443 abertos).
2	Dimensionar disco: ~1,84 GB/dia × 30 dias de retenção ≈ 55 GB; 80 GB recomendados.
3	Habilitar VPC peering do Lightsail e liberar a porta 5432 no security group do RDS apenas para o CIDR do Lightsail.
4	Instalar Node 22, git, Nginx e swap.
5–6	Clonar o repositório, npm ci, configurar server/.env, chave SFTP (chmod 600) e CA do RDS; validar com testar-conexao.
7	Rodar migrate, seed e criar o primeiro administrador.
8	Compilar o console (npm run build --workspace @infoprice/web).
9	Subir com PM2 (uma única instância), pm2 save/startup e logrotate.
10–11	Configurar Nginx, apontar DNS e emitir SSL com certbot (redirecionando HTTP para HTTPS).
12	Conferir /api/saude ("ok": true e conexão cifrada) e entrar no console.

Importante. Com `NODE_ENV=production`, o cookie de sessão sai marcado `Secure` e o navegador não o devolve por HTTP: o login parece funcionar e todo o resto responde 401. Use HTTPS, ou `COOKIE_SEGURO=false` apenas para teste temporário.

Disco efêmero. A área de spool guarda as cópias locais usadas por "Baixar bruto" e pelo download em .tar. Em contêiner (ECS/Fargate) esse disco é efêmero: monte EFS, migre para S3 ou reduza a retenção local. No Lightsail o disco é persistente.

Diagnóstico de problemas

Sintoma	Onde olhar
pm2 list mostra "errored"	pm2 logs console-ingestao --err --lines 50
Timeout ao conectar no RDS	VPC peering e security group
502 Bad Gateway	O Node caiu: pm2 list e logs
Login OK, mas tudo 401	COOKIE_SEGURO=true sem HTTPS
EADDRINUSE na 3001	pm2 delete all e subir de novo
Certbot não emite	DNS ainda não propagou (dig)
Disco cheio	Spool acima da retenção; revisar RETENCAO_LOCAL_DIAS ou anexar storage
Coleta recusada com 409	Já existe uma coleta em andamento; aguarde o término

Atualização

```
cd /opt/console-ingestao
git pull && npm ci
npm run build --workspace @infoprice/web        # se o front mudou
npm run migrate --workspace @infoprice/server   # se houver migração nova
pm2 restart console-ingestao
```

17. Glossário

Termo	Definição
Run	Um dia de coleta, identificado pela pasta run=AAAA-MM-DD na origem.
Execução	Uma passagem do pipeline sobre um run; um run pode ter várias (tentativas, reprocessamentos).
Staging	Tabela intermediária que espelha o arquivo como texto, antes da validação.
Fato	Tabela final de preços (fact_preco_coletado).
UPSERT / merge	Inserção que, em caso de chave existente, atualiza a linha.
Chave de conflito	gtin + id_loja + data_coleta + fonte: identifica um preço único.
Watermark	Data do último run efetivado com sucesso para a fonte.
Idempotência	Reexecutar o mesmo run produz o mesmo resultado, sem duplicar ou inflar contagens.
Linhagem	Caminho do dado desde o arquivo de origem até a linha gravada, e vice-versa.
Spool	Área local onde os arquivos baixados ficam guardados (cópia local).
GTIN	Código de barras global do produto (EAN), validado pelo dígito verificador GS1.
Incidente	Registro de uma condição de alerta, com severidade, run e resolução.